

ECE0202 Lecture 13: Interrupts 中文逐页讲义

Embedded Processors & Interfacing + Lab

Source: ECE0202_Lecture13_Interrupts.pdf, 55 slides.

核心逻辑

本讲的主线不是“背寄存器名字”，而是理解一个事件如何打断主程序、如何跳到 ISR、如何保存现场、如何返回现场。顺序是：**Polling vs Interrupt** → **Vector Table** → **Stacking/Unstacking** → **MSP/PSP 与 Handler Mode** → **ISR 调用子程序时 LR 被覆盖的问题与修正**。

Slide 1 - Title

关键词与中文注释

Interrupt (中断); Embedded Processor (嵌入式处理器); Interfacing (接口/外设交互)

这一页是标题页。本讲讨论 Cortex-M4 的中断机制。你要建立一个基本判断：**中断不是普通函数调用**。普通函数调用由程序自己执行 BL 进入；中断由硬件事件触发，处理器自动保存部分寄存器，然后跳到指定的 ISR。

Slide 2 - Interrupt vs Polling: Polling

关键词与中文注释

Polling (轮询/不断检查); **Busy Wait** (忙等); **Main Program** (主程序); **Sleep State** (睡眠状态)

核心逻辑

Polling 的本质: CPU 主动、反复地检查“事件是否发生”。类比就是你每隔几秒拿起手机看有没有来电。

- 优点: 逻辑简单, 初学者容易写; 例如在 `while(1)` 中反复读取 GPIO 输入寄存器。
- 缺点: 浪费 CPU 时间。事件没来时, CPU 仍在不停检查, 这就是 **busy wait** (忙等)。
- 嵌入式系统中, 轮询会增加功耗。真正的低功耗设计通常希望 CPU 没事时运行主程序或进入 **sleep state** (睡眠状态)。

易错点

不要把 polling 理解成“不能用”。简单实验中可以用, 但一旦事件不确定、响应时间重要、功耗重要, 轮询就开始显得笨重。CPU 一直盯门铃, 不等于系统聪明。

Slide 3 - Interrupt vs Polling: Interrupt

关键词与中文注释

Interrupt (中断); **Hardware-triggered Software Action** (硬件触发的软件动作); **External Event** (外部事件); **Avoid Busy Wait** (避免忙等)

核心逻辑

Interrupt 的本质：CPU 不主动反复检查事件，而是事件来了以后由硬件通知 CPU，CPU 暂停当前程序，跳转去执行中断服务程序。

- 中断的价值是**效率**：没事件时继续主程序或睡眠；事件来了再处理。
- 中断的触发源可以是外部引脚、定时器、串口接收、RTC alarm 等。
- 中断处理函数通常叫 **ISR** (**Interrupt Service Routine**, **中断服务程序**) 或 **IRQHandler**。

易错点

中断不是“并行线程”。在单核 Cortex-M4 上，中断只是改变执行流。它看起来像突然插队，但本质仍是同一个 CPU 在不同代码之间切换。

Slide 4 - Interrupt Vector Table

关键词与中文注释

Interrupt Vector Table (中断向量表); **NVIC** (Nested Vectored Interrupt Controller, 嵌套向量中断控制器); **ISR Address** (中断服务程序地址); **EXTI** (External Interrupt/Event Controller, 外部中断/事件控制器)

核心逻辑

中断向量表就是一张“中断号 → ISR 地址”的查表结构。中断号来了，处理器通过表找到该跳转到哪一个函数。

- 例子：PA3 触发 EXTI3，中断控制器识别出是 EXTI3，然后处理器根据向量表跳到 EXTI3_IRQHandler。
- 表中每个 entry 存一个 32-bit 地址，也就是对应 ISR 的入口地址。
- **NVIC** (嵌套向量中断控制器) 负责中断使能、优先级、pending/active 状态以及中断嵌套。

易错点

别把向量表理解成代码本身。向量表里放的是**地址**，真正执行的是地址指向的 ISR 代码。

Slide 5 - Memory Map of Cortex-M4

关键词与中文注释

Memory Map (内存映射); **Code Region** (代码区); **SRAM** (静态随机存取存储器); **Peripheral Region** (外设区); **System Region** (系统区)

这一页回顾 Cortex-M4 的 4GB 地址空间。重点不是背每段大小，而是理解中断相关的系统控制结构放在 **System region**。

- 0x00000000 附近通常映射启动入口、向量表等。
- 0x20000000 开始是 SRAM，用来放 heap、stack、运行时数据。
- 0x40000000 附近是外设寄存器，例如 GPIO、timer、UART 等。
- 0xE0000000 附近是系统控制区，包括 NVIC、SysTick、SCB 等。

易错点

中断不是“魔法跳转”。它依赖固定的 memory map、固定的 vector table、固定的 NVIC 控制寄存器。硬件很死板，错一个地址就不会按你想的运行。

Slide 6 - Instruction Memory and Vector Table Placement

关键词与中文注释

Instruction Memory (指令存储器); **Flash** (闪存); **Text Section** (代码段); **RO Data** (只读数据); **RW Data** (读写数据); **Initial MSP** (初始主栈指针)

这一页说明 STM32L4 的内部 Flash 通常从 0x08000000 开始, 而 0x00000000 地址处可以通过 aliasing 映射到 Flash 起始区域。启动时最关键的两个值是:

1. 0x00000000: 初始 **MSP** (**Main Stack Pointer**, 主栈指针) 的值。
 2. 0x00000004: **Reset_Handler** (复位处理函数) 的地址, 也就是初始 **PC** (**Program Counter**, 程序计数器)。
- 向量表位于 instruction memory 的开头。其后才是实际的 **text section** (代码段)、**RO data** (只读数据段)、**RW data** (读写数据段) 等。

Slide 7 - ISR Vector Table

关键词与中文注释

Top_of_Stack (栈顶初值); **Reset_Handler** (复位处理程序); **NMI** (不可屏蔽中断); **HardFault** (硬错误异常); **SysTick_Handler** (系统滴答定时器中断处理程序); **IRQHandler** (中断请求处理函数)

核心逻辑

向量表的第 0 项不是 ISR 地址, 而是初始 SP; 第 1 项才是 Reset_Handler 地址。很多人一开始会把这里记错。

- 0x00000000: Top_of_Stack, 用于初始化 SP。
- 0x00000004: Reset_Handler, 用于初始化 PC, 让程序从复位处理函数开始跑。
- 后续是系统异常: NMI_Handler、HardFault_Handler、SVC_Handler、PendSV_Handler、SysTick_Handler。
- 再往后是外设中断: WWDG_IRQHandler、EXTI0_IRQHandler、DMA1_Channel1_IRQHandler 等。

易错点

函数名字必须和启动文件/向量表中的弱符号匹配。你写了一个拼错名字的 ISR, 编译器不会替硬件猜你的意思。

Slide 8 - Interrupt Overview

这一页是过渡页。它引出后面最重要的机制：当中断发生时，处理器不能直接跳进 ISR，因为主程序的现场会丢。因此硬件要先执行 **stacking**（入栈保存现场），ISR 结束时再执行 **unstacking**（出栈恢复现场）。

Slide 9 - Stacking & Unstacking

关键词与中文注释

Stacking (自动入栈/保存现场); **Unstacking** (自动出栈/恢复现场); **Full Descending Stack** (满递减栈); **xPSR** (程序状态寄存器组合); **LR** (Link Register, 链接寄存器); **PC** (Program Counter, 程序计数器)

核心逻辑

中断进入前, Cortex-M 硬件自动把 8 个寄存器压入主栈: R0, R1, R2, R3, R12, LR, PC, xPSR。中断返回时, 硬件自动弹出这 8 个值。

栈帧布局如下:

相对地址	内容	含义
SP+0x00	R0	通用寄存器, 可能保存参数/临时值
SP+0x04	R1	通用寄存器
SP+0x08	R2	通用寄存器
SP+0x0C	R3	通用寄存器
SP+0x10	R12	intra-procedure-call 临时寄存器
SP+0x14	LR	中断前的返回链接信息
SP+0x18	PC	中断前下一条要执行的指令地址
SP+0x1C	xPSR	条件标志和执行状态

易错点

硬件只自动保存这 8 个寄存器。R4-R11 不是自动保存的。ISR 如果改了需要保留的寄存器, 必须自己保存, 别指望硬件替你擦屁股。

Slides 10-34 - Single Interrupt 动画序列

关键词与中文注释

Pending Register (挂起寄存器); **Active Register** (活动寄存器); **Enable Register** (使能寄存器); **Priority Register** (优先级寄存器); **BX LR** (跳转到 LR 指定地址/中断返回触发点)

这组页是同一个动画，不适合逐页机械拆开，否则你看到的是一堆重复图。它表达的是一个单次 EXTI3 中断从触发到返回的完整过程。

Slides 10-11: 中断条件已经具备

- 外设事件源是 EXTI3，对应中断号在图中用 9 表示。
- **Enable Register** (使能寄存器) 中对应位为 1，说明这个中断允许进入 NVIC。
- **Pending Register** (挂起寄存器) 会在事件发生后置位，表示“这个中断请求正在等待处理”。
- **Priority Register** (优先级寄存器) 决定多个中断同时出现时谁先执行。

Slides 12-21: Stacking 逐步发生

- 处理器先暂停当前主程序。
- SP 按 full descending stack 的规则向低地址移动。
- 硬件依次保存 xPSR, PC, LR, R12, R3, R2, R1, R0，最终形成一个 8-word 的栈帧。
- 保存 PC 的意义：中断返回时知道主程序该从哪里继续。
- 保存 xPSR 的意义：恢复条件标志、Thumb 状态等执行状态。

Slides 22-23: ISR 执行并遇到 BX LR

- Stacking 完成后，处理器根据向量表跳到 EXTI3_IRQHandler。
- ISR 执行期间，程序处于 **Handler Mode** (处理器异常处理模式)。
- ISR 末尾执行 BX LR。在普通函数里它表示“跳回调用者”；在 ISR 里，LR 通常是特殊的 **EXC_RETURN** (异常返回值)，例如 0xFFFFFFF9，用于触发硬件 unstacking。

Slides 24-33: Unstacking 逐步发生

- 执行 BX LR 后，如果 LR 是有效的 EXC_RETURN，硬件开始把栈帧弹回寄存器。
- 弹出顺序恢复为 R0, R1, R2, R3, R12, LR, PC, xPSR。
- SP 最终回到中断进入前的位置。
- **Active Register** (活动寄存器) 对应位清除，表示 ISR 已结束。
- 主程序恢复执行，好像中断只是中间插了一段短代码。

Slide 34: 主程序恢复

动画最后回到 **Thread Mode** (线程模式/普通程序模式)。关键结论：中断处理的正确性靠三件事保证：向量表给出入口、硬件保存现场、EXC_RETURN 触发恢复现场。

易错点

中断返回不是普通意义上的“跳转”。BX LR 看到的是特殊 LR 值，它会让硬件执行异常返回流程。如果你在 ISR 里把 LR 覆盖掉，又不保存，返回机制就断了。

Slide 35 - Stacking & Unstacking Summary

关键词与中文注释

Thread Mode (线程模式/普通程序模式); **Handler Mode** (处理器模式/中断异常模式); **Interrupt Signal** (中断信号); **Interrupt Exit** (中断退出)

这一页把上面的动画压缩成时间线:

User Program → Stacking → Interrupt Handler → Unstacking → User Program

- 主程序运行在 **Thread Mode** (线程模式)。
- 中断处理程序运行在 **Handler Mode** (处理器/异常处理模式)。
- 进入 Handler Mode 前自动 stacking; 退出 Handler Mode 时自动 unstacking。

易错点

“中断时 CPU 继续跑主程序” 是错误理解。正确理解是: 主程序被暂停, 中断代码插队执行, 结束后主程序继续。

Slide 36 - Registers: MSP and PSP

关键词与中文注释

MSP (Main Stack Pointer, 主栈指针); **PSP** (Process Stack Pointer, 进程栈指针); **CONTROL Register** (控制寄存器); **Privileged** (特权级); **Unprivileged** (非特权级)

Cortex-M 有两个硬件栈指针: MSP 和 PSP。R13 (SP) 只是当前被选中的那个栈指针的影子。

- **Handler Mode** (中断/异常处理模式): $SP = MSP$ 。这点非常重要, ISR 默认使用主栈。
- **Thread Mode** (普通程序模式): 如果 $CONTROL[0]=0$, 使用 MSP; 如果 $CONTROL[0]=1$, 使用 PSP。
- 操作系统或 RTOS 常用 PSP 给任务线程使用, 用 MSP 给异常/内核使用。

易错点

本课目前多数裸机实验主要看到 MSP。PSP 不常手动用, 但必须知道它存在, 否则你看异常返回值和 RTOS 上下文切换时会彻底迷路。

Slide 37 - Register Values in an ISR

关键词与中文注释

EXC_RETURN (异常返回编码值); **LR=0xFFFFFFFF9** (使用 **MSP** 返回 **Thread Mode** 的典型异常返回值); **Handler Mode** (中断处理模式)

这一页强调: 进入 ISR 后, LR 不是普通函数返回地址, 而是一个特殊值。图中给出 $LR = 0xFFFFFFFF9$ 。

- $0xFFFFFFFF9$ 告诉处理器: 异常返回后回到 Thread Mode, 并使用 MSP 中保存的栈帧恢复现场。
- ISR 总是在 Handler Mode 执行, 因此 $SP = MSP$ 。
- 这也是为什么 ISR 中随便调用子程序会有坑: BL subroutine 会改写 LR, 可能覆盖这个 EXC_RETURN 值。

Slides 38-47 - SysTick 中断的具体寄存器与栈变化

关键词与中文注释

SysTick (系统滴答定时器); **Stack Frame** (栈帧); **R13(SP)** (栈指针); **R14(LR)** (链接寄存器); **R15(PC)** (程序计数器)

这组页用具体数值解释一次 SysTick_Handler 中断。初始情形大致是：主程序中 R3=3, R4=4, MSP=0x20000200, PC=0x08000044。

Slides 38-39: 中断发生前

- 主程序正在运行，假设 PC = 0x08000044 时 SysTick 中断发生。
- 此时 R0-R3, R12, LR, PC, xPSR 都属于主程序现场的一部分。

Slides 40-41: 硬件自动 stacking

- MSP 从 0x20000200 下降到 0x200001E0，因为保存 8 个 32-bit word，共 $8 \times 4 = 32$ bytes。
- 栈中保存：R0, R1, R2, R3, R12, LR, PC, xPSR。
- 进入 ISR 后，LR 被设置为 0xFFFFFFFF9，表示之后应通过 MSP 执行异常返回。
- PC 改为 SysTick_Handler 的入口地址，例如 0x0800001C。

Slides 42-44: ISR 修改寄存器

- ISR 中执行 ADD r3, #1，所以当前 R3 从 3 变为 4。
- 执行 ADD r4, #1，R4 从 4 变为 5。
- 注意差异：R3 在硬件栈帧里被保存；R4 没有被硬件自动保存。

Slides 45-47: unstacking 之后的结果

- 执行 BX lr，由于 LR=0xFFFFFFFF9，硬件开始 unstacking。
- R0-R3, R12, LR, PC, xPSR 从栈中恢复成中断前的值。
- 因此 ISR 中对 R3 的修改会丢失：它被恢复为中断前的 R3=3。
- R4 没有被硬件保存，所以 ISR 对 R4 的修改保留下来，主程序恢复后看到 R4=5。

易错点

这页的“Note the new value of R3 is lost!!!”是全讲最关键的提醒之一：硬件自动保存的寄存器，在 ISR 结束后会恢复旧值；你在 ISR 里改它们，除非把结果写到内存/全局变量/外设寄存器，否则修改不会影响主程序。

Slide 48 - What if ISR Calls Subroutine?

关键词与中文注释

Subroutine (子程序/函数); **BL** (Branch with Link, 带链接跳转); **LR Overwrite** (LR 被覆盖)

这一页提出真正危险的问题: **如果 ISR 内部调用另一个函数, 会发生什么?**

核心逻辑

普通函数调用使用 **BL** `sine`。BL 的硬件动作之一是把返回地址写入 LR。但 ISR 的 LR 原本保存的是 **0xFFFFFFFF9** 这种异常返回值。一旦被覆盖, **BX** LR 就不能触发正确的 `unstacking`。

Slides 49-54 - ISR 调用子程序但没有保存 LR 的错误过程

关键词与中文注释

Nested Call (嵌套调用); **LR Clobbering** (LR 被破坏/覆盖); **Exception Return** (异常返回); **Return Address** (普通返回地址)

这组页展示一个典型 bug: SysTick_Handler 里执行 BL sine, 但是没有先保存 LR。

1. 进入 ISR 时, 硬件把主程序现场保存到 MSP 栈帧, 并设置 LR=0xFFFFFFFF9。
2. ISR 执行 ADD r4, #1, 然后执行 BL sine。
3. BL sine 会把普通函数返回地址写入 LR, 例如 0x08000024。
4. 这会覆盖掉原来的 0xFFFFFFFF9。
5. 子程序 sine 返回后, ISR 末尾执行 BX lr, 但此时 LR 不再是 EXC_RETURN, 而是一个普通地址。
6. 结果: **unstacking** (自动恢复现场) 不会发生, 程序流可能跳错、死循环或状态破坏。

易错点

这不是小瑕疵, 是结构性错误。ISR 中只要使用 BL 调子程序, 就必须先保护 LR。否则异常返回值丢失, 硬件就不知道该恢复主程序现场。

Slide 55 - Correct Method: Save LR in ISR

关键词与中文注释

PUSH {lr} (把 LR 压栈保存); **POP {pc}** (弹出到 PC, 相当于返回); **Callee-save** (被调用者保存); **Manual Save/Restore** (手动保存/恢复)

这一页给出正确方向：在 ISR 调用子程序前，先保存 LR。

核心逻辑

安全写法的核心是：先 **PUSH {lr}** 保存 0xFFFFFFFF9，调用子程序后再恢复它，最后用恢复出来的 **EXC_RETURN** 触发中断返回。

典型结构如下：

```

SysTick_Handler PROC
    EXPORT SysTick_Handler
    PUSH {lr}          ; 保存 EXC_RETURN，防止 BL 覆盖 LR
    ADD    r4, #1
    BL     sine         ; BL 会改 LR，但原始 LR 已经保存在栈里
    POP    {pc}         ; 弹出原来的 EXC_RETURN 到 PC，触发异常返回
ENDP

```

或者写成更显式的形式：

```

    PUSH {lr}
    BL    sine
    POP   {lr}
    BX    lr

```

易错点

POP {pc} 在这里不是普通“取回一个地址”那么简单。如果弹出来的是 0xFFFFFFFF9 这种 **EXC_RETURN**，处理器会执行异常返回流程并 unstacking。你看到的是一条指令，背后是硬件状态机在工作。

总复习: Lecture 13 必须掌握的 10 个点

1. **Polling vs Interrupt:** 轮询是 CPU 主动查; 中断是硬件事件触发 CPU 响应。
2. **Vector Table:** 向量表存的是 ISR 地址, 不是 ISR 代码。
3. **第 0 项和第 1 项:** `0x00000000` 是初始 SP, `0x00000004` 是 Reset_Handler 地址。
4. **NVIC:** 管中断使能、优先级、pending、active、嵌套。
5. **自动 stacking:** 硬件保存 R0-R3, R12, LR, PC, xPSR。
6. **自动 unstacking:** 异常返回时硬件恢复这些寄存器。
7. **Handler Mode:** ISR 总是在 Handler Mode, 使用 MSP。
8. **LR=0xFFFFFFF9:** 这是异常返回值, 不是普通函数返回地址。
9. **ISR 改 R3 会丢:** 因为 R3 被硬件保存并恢复; 要保留结果应写内存或全局变量。
10. **ISR 调子程序必须保存 LR:** 否则 BL 覆盖 EXC_RETURN, unstacking 不会正确发生。

最容易考的判断题/简答题

- **Q: 中断发生时, 哪些寄存器自动入栈?**
A: R0, R1, R2, R3, R12, LR, PC, xPSR。
- **Q: ISR 中修改 R3, 主程序一定能看到吗?**
A: 不一定。若只是改寄存器, unstacking 会恢复旧 R3, 修改丢失。
- **Q: ISR 中调用普通子程序为什么要 PUSH {lr}?**
A: 因为 BL 会覆盖 LR, 而 ISR 的 LR 保存的是异常返回编码值。
- **Q: Handler Mode 用 MSP 还是 PSP?**
A: MSP。
- **Q: BX LR 在 ISR 里一定是普通函数返回吗?**
A: 不是。如果 LR 是 EXC_RETURN, BX LR 会触发异常返回和 unstacking。